

ContextRail Hero Demo：工程決策報告包

示例資料 / 非真實部署證據

這份文件是用來展示 ContextRail 的報告產出，不代表目前已經連到真實專案、Cloud Run、Cloud Build 或企業內部文件。所有 commit、digest、hash、成本與測試狀態均為 fixture，不能用來宣稱已完成部署或已通過 production gate。

這份報告要回答什麼

同一個變更不應該只產生一段 AI 摘要。企業需要的是一份可追溯的 Engineering Decision Record，再由同一份來源產生不同讀者看得懂、也能執行的視圖：

- 主管 / PM：到底要不要做、會影響什麼、現在需要誰決定。
- Solution Architect / Tech Lead：架構、服務、成本、風險與替代方案。
- Developer / AI Agent：可以改哪些檔案、禁止做什麼、驗收條件。
- DevOps / SRE：如何驗證 build、digest、設定與 staging rollout。
- FDE / Implementation：客戶需要補什麼資料、如何解釋限制與下一步。

若來源資料不足，報告會產生 `NEEDS_REVIEW` 與缺資料清單，不會自行補齊不存在的事實，也不會發出可執行 Work Order。

0. Canonical Engineering Decision Record

0.1 變更識別

欄位	示例值
<code>decision_id</code>	CHG-2026-0017
<code>decision_version</code>	3
變更類型	既有內部系統新增功能
產品情境	Internal Approval System 新增申請附件
目前狀態	<code>APPROVED_FOR_STAGING_ONLY</code>
<code>source_snapshot_hash</code>	sha256:example-source-snapshot-7f83...
<code>policy_version</code>	internal-cloudrun-v1
決策人	Business Owner (示例)
技術負責人	Solution Architect (示例)
有效期限	2026-09-25T23:59:59+08:00
真實證據狀態	<code>NOT_EXECUTED</code> ；本文件只展示格式

0.2 Project 環境拓撲與本次 promotion 邊界

順序	Project 環境	標準類型	這次允許的動作	必要證據	狀態
1	Development	development	建立 feature branch、執行本機檢查	Work Order、changed paths	已定義
2	Testing	testing	執行整合測試	PR、unit / integration tests、build provenance	已定義
3	Staging	staging	部署已接受 digest、執行 smoke	Candidate Decision、independent review、config hash	本次目標

順序	Project 環境	標準類型	這次允許的動作	必要證據	狀態
4	Production	production	只能請求 promotion	Staging verified、 Release Approval、 backup evidence	P0 blocked

環境名稱由 Project owner 定義，但 Production 的最低保護不可透過改名繞過。Code Review、Candidate Review 與 Release Approval 是 gate，不是環境；本示例只展示環境治理 metadata，不代表已執行任何真實部署。

0.2 業務需求

目前員工送出簽核申請後，只能在外部系統貼上文件連結。需求是讓員工在申請單上上傳附件，讓審核者在相同的內部系統中查看附件，並保留基本的存取紀錄。

本次要解決的是「申請資料與附件分離造成的審核遺漏」，不是建置一個通用檔案管理平台。

0.3 已確認的需求與限制

類別	已確認內容	來源 / 責任人
使用者	申請人可上傳；同一申請的審核者可查看	Product Owner (示例)
檔案大小	單檔最多 25 MB	Product Owner (示例)
檔案格式	PDF、DOCX、XLSX、PNG、JPG	Product Owner (示例)
保留期限	365 天，期滿自動刪除	Compliance Owner (示例)
預估流量	每月 10,000 次上傳、30,000 次下載	Product Owner (示例)
資料分類	Internal Confidential	Security Owner (示例)
地區	asia-east1	Solution Architect (示例)
發佈邊界	只允許先到 staging；本次不允許 production	Release Owner (示例)

0.4 需求結論

在示例資料已補齊的前提下，選擇「Cloud Run 產生短效 signed URL，由 Cloud Storage 直接傳輸檔案」作為 staging candidate。Cloud Run 仍然負責身分驗證、申請單授權、檔案 metadata、稽核事件與 URL 產生；Storage bucket 不允許 public access。這是「建議可進入 staging 驗證」，不是「已批准 production」。如果 source snapshot、政策版本、架構限制或關鍵需求改變，decision_version 必須增加，舊的 Agent Context Pack 與 Work Order 立即失效。

1. 主管 / PM 版：Change Decision Brief

一句話結論

可以做，但先以 private Cloud Storage + 短效 signed URL 在 staging 驗證；production 上線前仍必須補齊真實成本、資安審查、備份與 rollout 證據。

為什麼值得做

附件與簽核單放在同一個流程裡，可以減少審核者找檔案、權限錯誤與申請資料遺漏。這個功能也能驗證公司未來使用 Cloud Run、Cloud Storage 與 AI 輔助交付時，是否能保留決策、責任與證據鏈。

對業務的影響

問題	預期改善	需要接受的代價
審核者找不到附件	在申請單內直接查看	需要定義檔案格式與保留期限

問題	預期改善	需要接受的代價
附件權限不一致	沿用申請單的角色授權	需要測試申請人、審核者、非關係人三種角色
需求變更容易直接進開發	先產出決策與缺資料報告	初期會比直接請 AI 寫 code 多一個 gate
AI 產出難以稽核	每個視圖共享 <code>decision_id</code> 與版本	必須維護 source snapshot 與 evidence refs

本次要決定的事

決策	建議	決策人	狀態
是否採用 signed URL	採用，限制為短效、private bucket	Solution Architect + Security Owner	示例已決定
是否進 staging	可以	Release Owner	示例已決定
是否進 production	不在本次範圍	Production Approver	明確拒絕 / 未授權
是否建立通用檔案平台	不做	Product Owner	明確拒絕

不做會怎樣

如果不做，原本的外部連結流程仍可運作，但審核體驗、授權一致性與稽核完整性不會改善。這不是必做的緊急修復，而是有明確業務收益、可以用 staging 控制風險的增量變更。

下一步

- 1. Product Owner 確認需求與保留期限。
- 2. Security Owner 確認 Internal Confidential 的檔案處理政策。
- 3. Developer 只依照 Agent Context Pack 修改允許路徑。
- 4. CI 產生測試、artifact digest 與 build provenance。
- 5. 先部署到 staging，完成角色、逾期刪除與拒絕案例驗證。

2. Solution Architect / Tech Lead 版：架構與成本報告

2.1 現況與目標架構

現有系統是 Go API 部署在 Cloud Run，申請單 metadata 放在 Firestore。新增附件後，建議的資料流如下：



2.2 方案比較

方案	優點	主要風險	示例判斷
Cloud Run proxy 轉送檔案	授權與資料流集中、容易先做	大檔案佔用 Cloud Run 流量與 request time；成本與 timeout 風險較高	可作 fallback，不選主方案
Signed URL 直傳 Storage	Cloud Run 不承載檔案內容；容易限制有效期與 private access	URL 管理、CORS、撤銷與 audit 必須設計清楚	選為 staging candidate

方案	優點	主要風險	示例判斷
直接讓前端存取 public bucket	開發看似簡單	資料外洩、無法滿足 Internal Confidential	禁止

2.3 服務與責任邊界

元件	本次用途	變更影響	本次是否建立
Cloud Run	API、授權、signed URL、metadata	timeout、service account、request log	使用既有服務
Firestore	附件 metadata 與 audit event	schema、index、讀寫權限	增加 collection / 欄位
Cloud Storage	private 檔案內容與 lifecycle	bucket policy、CORS、retention	建立 staging bucket
Cloud Build	build、test、provenance	pipeline step、artifact digest	使用既有 pipeline
Artifact Registry	儲存 container image	image tag / digest	使用既有 registry
Cloud Deploy	staging rollout / approval	target、approval gate	本次只驗證 staging
Cloud KMS	CMEK	key rotation、IAM、成本	MVP 不引入，需另案決策

2.4 安全與可靠性控制

- bucket 必須保持 private，禁止 public IAM binding。
- signed URL 有效期預設 5 分鐘，URL 不作為長期權限。
- API 先驗證使用者與申請單角色，再產生 upload 或 download URL。
- metadata 不存檔案內容；檔案 path 使用不可猜測的 `request_id/attachment_id`。
- lifecycle rule 在 365 天後刪除物件；刪除動作需要有可查詢的 audit event。
- staging 與 production 使用不同 bucket、service account 與設定快照。
- production 不接受僅有 tag 的 image；必須綁定 immutable digest。
- 檔案病毒掃描、DLP、CMEK 若是政策要求，必須先升級成新的決策版本，不可默默塞入 agent scope。

2.5 成本模型（示例，不是報價）

成本驅動因子	示例輸入	對成本的影響	目前狀態
新增儲存量	10,000 × 平均 5 MB / 月	Storage 增長與 365 天保留期	已提供；需用真實 distribution 校正
下載流量	30,000 次 / 月	egress、signed URL 下載流量	未提供來源地區，不能做 production 報價
Cloud Run request	產生 URL 與 metadata	request、CPU、memory	可沿用基線，需用 staging 觀測校正
Firestore reads/writes	每次上傳 / 下載的 metadata	operation 數量與 index	需用實際 query pattern 驗證
Build / Deploy	每次 PR、staging rollout	build minutes、artifact storage	可由既有 pipeline 估算

成本結論不是「每月一定 USD X」。這個示例只允許輸出：**成本可估算，但真實金額仍取決於流量、檔案平均大小、下載地區、保留量與當期 pricing calculator**。這樣的報告比硬填一個看似精準的數字更適合企業決策。

2.6 非功能驗收條件

類別	staging 驗收條件
授權	申請人可上傳自己的附件；審核者可查看；非關係人收到 403
整合	signed URL 不暴露 bucket public access；metadata 與申請單關聯正確
安全	static policy 檢查拒絕 public bucket、過長 URL TTL、未授權路徑

類別	staging 驗收條件
保留	365 天 lifecycle 規則存在且可由設定快照證明
可靠性	失效 URL、重複 upload、超過大小、錯誤申請單均有明確錯誤
可觀測性	request、attachment、actor、decision_id 可串成 audit trail
發佈	staging image digest、build provenance、config hash 三者可對應

3. Decision Readiness 版：缺資料時怎麼停止

這一份是同一需求在尚未補齊資料時的輸出。它故意保留阻擋狀態，展示系統不應該為了讓報告看起來完整而自行捏造答案。

3.1 初始輸入狀態

缺少的資料	必要性	應由誰提供	沒有它會影響什麼	狀態
retention days	必要	Compliance Owner	lifecycle、成本、刪除政策	MISSING
每月上傳 / 下載量	必要	Product Owner	Storage、egress、Firestore 成本	MISSING
單檔大小與格式	必要	Product Owner	Cloud Run proxy 或 signed URL 的方案判斷	PARTIAL
資料分類與地區	必要	Security Owner	bucket region、DLP、加密與存取政策	MISSING
審核者與非關係人權限	必要	Product Owner	authorization rule 與測試案例	MISSING
RTO / RPO	production 必要	SRE / Service Owner	backup、recovery 與 production gate	MISSING

3.2 缺資料時的決策結果

Gate	結果
decision_readiness	NEEDS_REVIEW
work_order_issuance	BLOCKED
production_deploy	BLOCKED
AI 可做的事	產生問題清單、列出需要的證據、比較暫定方案
AI 不可做的事	自行假設 retention、替公司選 region、把暫定成本當承諾、發出 deploy 指令

3.3 補資料後為什麼要升版

在這個示例中，Business Owner 後來補上 365 天保留、每月 10,000 次上傳、30,000 次下載；Security Owner 確認 Internal Confidential 與 `asia-east1`。因此從 version 2 產生 version 3，重新計算 `source_snapshot_hash`，重新產生人類報告與 Agent Context Pack。

舊版本的 Work Order 不會自動復活。這是為了避免 agent 仍依照已過期的假設修改程式或部署資源。

4. Developer / AI Agent 版：Agent Context Pack

這一份不是給主管閱讀的長報告，而是受到邊界限制的機器契約。完整 JSON 範例在同一資料夾的

`AGENT_CONTEXT_PACK_HERO.example.json`。

4.1 Agent 被允許做什麼

- 修改 `internal/attachments/**`、`internal/storage/**`、`tests/attachments/**`。
- 實作 attachment metadata、signed URL service、角色授權與測試。
- 更新必要的 API schema 與 staging-only 設定範本。
- 在本地或受控 CI 執行 unit test、integration test、static policy test。

4.2 Agent 被禁止做什麼

- 不可修改任意 IAM policy、建立 public bucket 或讀取 deployment secret。
- 不可部署 production、變更 production database schema、替換 release approval。
- 不可把 `MISSING` 欄位自行當成已確認需求。
- 不可將新的需求擴大成通用檔案平台、病毒掃描平台或跨租戶 SaaS。

4.3 Agent 的完成條件

Agent 只有在以下條件全部達成時，才可以回報「implementation candidate ready」：

1. 所有變更都在 allowed paths。
2. 上傳、下載、超過大小、失效 URL、未授權角色測試有結果。
3. public bucket、過長 URL TTL、production path 變更的 policy test 有結果。
4. 回報 `decision_id`、source snapshot、changed paths、test commands 與結果。
5. 沒有自行處理 out-of-scope 或 missing inputs。

「測試通過」不等於「可以上 production」。Agent 只能回報證據；merge、approval、deploy 仍然是人與既有 release workflow 的權限。

5. DevOps / SRE 版：Evidence Bundle 與 Release Receipt

5.1 必須收集的證據

證據	用途	示例狀態
source commit / PR	確認變更來源	<code>FIXTURE_ONLY</code>
changed-path manifest	確認沒有越界	<code>FIXTURE_ONLY</code>
unit / integration test result	確認功能與角色行為	<code>NOT_EXECUTED</code>
policy test result	確認 public access、TTL、production path	<code>NOT_EXECUTED</code>
container image digest	確認部署不可變 artifact	<code>FIXTURE_ONLY</code>
build provenance	確認 image 的來源與建置資訊	<code>NOT_EXECUTED</code>
staging config hash	確認服務、bucket、region 與設定	<code>FIXTURE_ONLY</code>
rollout / approval event	確認誰在何時批准	<code>NOT_EXECUTED</code>
smoke test	確認 staging 真實路徑	<code>NOT_EXECUTED</code>

5.2 Release Receipt (示例)

```
receipt_id: RR-2026-0017-STG-001
receipt_status: SIMULATED_NOT_EXECUTED
decision_id: CHG-2026-0017
decision_version: 3
environment: staging
artifact_digest: sha256:example-image-digest-91bc...
source_snapshot_hash: sha256:example-source-snapshot-7f83...
config_snapshot_hash: sha256:example-staging-config-allc...
build_provenance: REQUIRED_BUT_NOT_ATTACHED
approval_event: REQUIRED_BUT_NOT_ATTACHED
smoke_test: NOT_EXECUTED
production_authorized: false
```

這個 receipt 的重點不是「看起來像成功」，而是把能證明什麼、還缺什麼清楚分開。只要 digest、source snapshot 或 config hash 對不起來，release gate 應該回到 `BLOCKED`。

6. 一個越界案例：為什麼不是所有 AI 產出都能直接合併

假設 team member 的 feature branch 額外修改：

```
infra/iam/public-bucket.yaml
```

並新增 `allUsers: objectViewer`。即使 attachment API 的 unit test 全部通過，ContextRail 仍應輸出：

欄位	結果
candidate status	BLOCKED
blocking reason	變更觸碰 forbidden path 且可能暴露 Internal Confidential 檔案
可否繼續測試	可以做隔離的靜態檢查，但不能產生可部署 Work Order
需要誰決定	Security Owner + Solution Architect
是否沿用原決策	不可；必須建立新的 decision version 或退回該變更

這個案例也說明「feature branch」不是問題本身。問題是每個 branch 的 changed paths、commit、測試與 decision version 是否仍然落在同一條可追溯證據鏈內。

7. Single Source of Truth 規則

所有角色看到的文件都必須共享以下識別欄位：

共用欄位	作用
decision_id	找到同一個需求變更
decision_version	知道自己依照哪一版決策
source_snapshot_hash	追溯輸入文件、需求與架構快照
policy_version	知道採用哪一組規則
evidence_refs	由報告連到 PR、測試、provenance、rollout 證據
approver / expiry	確認誰批准、何時失效

以下任一事件發生時，所有衍生視圖都要標成 stale，不能靜默沿用：

- 需求、保留期限、資料分類或流量假設改變。
- 架構方案、Cloud Run service、bucket、region 或 IAM 邊界改變。
- source snapshot 或 policy version 改變。
- agent scope、allowed paths、forbidden actions 改變。
- approval 過期或 evidence bundle 出現 digest / config drift。

8. 看完這份報告後，使用者實際得到什麼

不是單——篇 AI 文章，而是一個可以放進企業 workflow 的決策包：

1. 一份給人讀的、格式一致的 Change Decision Brief。
2. 一份給架構與成本審查的 Technical Decision Report。
3. 一份在資料不足時敢於停止的 Decision Readiness Report。
4. 一份限制 agent 行為的 Agent Context Pack。
5. 一份把測試、digest、provenance、設定與 rollout 串起來的 Evidence / Release Receipt。

它的產品價值在於「決策、執行邊界與證據互相綁定」。報告是重要產出，但不是終點；真正的差異化是報告可以直接成為受控的下一步，而不是讓人再把 AI 摘要手動翻譯成工程行動。

附件

- `README.zh-TW.md`：角色化報告索引與 Single Source of Truth 規則。
- `CANONICAL_DECISION_RECORD_HERO.example.json`：所有角色視圖的唯一事實來源。
- `01_EXECUTIVE_CHANGE_DECISION_BRIEF_HERO.zh-TW.md`：主管 / PM 白話版。
- `02_TECHNICAL_ARCHITECTURE_COST_REPORT_HERO.zh-TW.md`：架構 / 成本版。
- `03_IMPLEMENTATION_READINESS_HERO.zh-TW.md`：FDE / Implementation 缺資料與下一步版。
- `AGENT_CONTEXT_PACK_HERO.example.json`：機器 / Agent 讀取的 Context Pack。
- `CANDIDATE_REVIEW_BLOCKED_HERO.example.json`：feature branch 越界時的阻擋結果。
- `RELEASE_RECEIPT_HERO.example.json`：DevOps / SRE 使用的 staging receipt。